

OSP Assignment 10 - Technical Report

컴퓨터공학과 2317022 이다영

** 실험결과는 레포트의 마지막 부분에 작성되어 있습니다.

vgg16_full.py

VGG는 3×3 conv 필터를 깊게 쌓아 표현력을 높인 모델이다. 본 과제에서 사용하는 VGG-16의 구조는 아래와 같다.

```
Input: 3 × 32 × 32 (CIFAR-10)
[3×3 Conv, 64] → [3×3 Conv, 64] → MaxPool
[3×3 Conv, 128] → [3×3 Conv, 128] → MaxPool
[3×3 Conv, 256] → [3×3 Conv, 256] → [3×3 Conv, 256] → MaxPool
[3×3 Conv, 512] → [3×3 Conv, 512] → [3×3 Conv, 512] → MaxPool
[3×3 Conv, 512] → [3×3 Conv, 512] → [3×3 Conv, 512] → MaxPool
→ Dropout → FC(512-512) → BN → ReLU → Dropout → FC(512-10)
```

`vgg16_full.py`에서는 위 구조에 따라 VGG-16을 구현하고, CIFAR-10 데이터셋에서 이미지 분류를 수행한다.

1. `class VGG(nn.Module)`

VGG 클래스는 VGG-16 모델의 전체 구조를 정의한다.

1.1 `__init__(self, features)`

```
super(VGG, self).__init__()
self.features = features
self.classifier = nn.Sequential(
    nn.Dropout(),
    nn.Linear(512, 512),
    nn.BatchNorm1d(512),
    nn.ReLU(True),
    nn.Dropout(),
    nn.Linear(512, 10),
)
```

`super().__init__()`으로 모듈을 초기화한다. `features`는 convolution layer들의 묶음이며, `classifier`는 FC layer 부분을 `nn.Sequential`로 정의한 것이다. 순전파 시 convolution 결과 벡터는 다음 과정을 거친다:

- Dropout
- 512차원 → 512차원 선형 변환
- Batch Normalization
- ReLU
- Dropout
- 512차원 → 10차원 선형 변환 (CIFAR-10 클래스 수)

```

for m in self.modules():
    if isinstance(m, nn.Conv2d):
        n = m.kernel_size[0] * m.kernel_size[1] * m.out_channels
        m.weight.data.normal_(0, math.sqrt(2. / n))
        m.bias.data.zero_()

```

이후 conv layer의 가중치를 He initialization 방식으로 초기화한다.

1.2 forward(self, x)

```

x = self.features(x)
x = x.view(x.size(0), -1)
x = self.classifier(x)
return x

```

`features`에 입력을 통과시켜 feature map을 생성하고, `view()`로 flatten한 뒤 `classifier`에 전달해 최종 클래스 점수를 반환한다.

2. make_layers(cfg, batch_norm=False)

`cfg`에 정의된 구조에 따라 convolution 블록들을 순서대로 생성한다.

```

layers = []
in_channels = 3

```

RGB 이미지를 사용하므로 입력 채널을 3으로 초기화한다.

```

for v in cfg:
    if v == 'M':
        layers += [nn.MaxPool2d(kernel_size=2, stride=2)]
    else:
        conv2d = nn.Conv2d(in_channels, v, kernel_size=3, padding=1)
        if batch_norm:
            layers += [conv2d, nn.BatchNorm2d(v), nn.ReLU(inplace=True)]
        else:
            layers += [conv2d, nn.ReLU(inplace=True)]
        in_channels = v
return nn.Sequential(*layers)

```

`v == 'M'`이면 `MaxPool2d`를 추가하고, 숫자이면 `Conv2d`를 생성한다. `batch_norm` 여부에 따라 `BatchNorm2d` 포함 여부가 결정된다. 매 iteration마다 `in_channels`를 갱신해 다음 레이어로 이어질 수 있게 한다.

3. vgg16()

```

cfg = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M',
512, 512, 512, 'M']
return VGG(make_layers(cfg))

```

사용할 레이어 구조를 `cfg` 에 정의하고 `make_layers()` 에 전달해 VGG 모델을 생성한다.

resnet50_full.py

ResNet은 skip connection을 통해 입력을 출력에 그대로 더할 수 있는 residual block을 사용함으로써, 레이어가 깊어질수록 발생하는 vanishing gradient 문제를 완화한 모델이다. `resnet50_full.py` 에서는 ResNet-50을 구현하고 CIFAR-10 데이터셋에서 이미지 분류를 수행한다.

1. `conv1x1(in_channels, out_channels, stride, padding)`

```
model = nn.Sequential(
    nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride, padding=padding),
    nn.BatchNorm2d(out_channels),
    nn.ReLU(inplace=True)
)
return model
```

1×1 convolution → Batch Normalization → ReLU로 구성된 블록을 정의한다. 1×1 conv는 spatial size를 유지하면서 채널 수를 변환하는 역할을 한다.

2. `conv3x3(in_channels, out_channels, stride, padding)`

```
model = nn.Sequential(
    nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=padding),
    nn.BatchNorm2d(out_channels),
    nn.ReLU(inplace=True)
)
return model
```

3×3 convolution → Batch Normalization → ReLU로 구성된 블록을 정의한다. `padding=1` 로 사용하면 spatial size를 유지할 수 있다.

Question 1: `class ResidualBlock(nn.Module)`

ResidualBlock은 skip connection을 포함하는 bottleneck 구조의 residual block을 정의한다.

3.1 `__init__(self, in_channels, middle_channels, out_channels, downsample=False)`

파라미터는 다음과 같다:

- `in_channels`: 입력 채널 수
- `middle_channels`: bottleneck 중간 채널 수
- `out_channels`: 출력 채널 수
- `downsample`: spatial size를 절반으로 줄일지 여부

`downsample=True` 인 경우:

```

self.layer = nn.Sequential(
    conv1x1(in_channels, middle_channels, stride=2, padding=0),
    conv3x3(middle_channels, middle_channels, stride=1, padding=1),
    conv1x1(middle_channels, out_channels, stride=1, padding=0)
)
self.downsize = conv1x1(in_channels, out_channels, 2, 0)

```

첫 번째 conv1×1에 `stride=2` 를 적용해 spatial size를 절반으로 줄인다. skip connection에 사용할 `downsize` 또한 `stride=2` 로 설정해 F(x)와 해상도를 일치시킨다.

`downsample=False` 인 경우:

```

self.layer = nn.Sequential(
    conv1x1(in_channels, middle_channels, stride=1, padding=0),
    conv3x3(middle_channels, middle_channels, stride=1, padding=1),
    conv1x1(middle_channels, out_channels, stride=1, padding=0)
)
self.make_equal_channel = conv1x1(in_channels, out_channels, 1, 0)

```

모든 conv에서 `stride=1` 로 spatial size를 유지한다. `in_channels != out_channels` 인 경우 skip connection의 채널 수를 맞추기 위해 `make_equal_channel` 을 정의한다.

3.2 `forward(self, x)`

```

if self.downsample:
    out = self.layer(x)
    x = self.downsize(x)
    return out + x
else:
    out = self.layer(x)
    if x.size() != out.size():
        x = self.make_equal_channel(x)
    return out + x

```

$H(x) = F(x) + x$ 형태로 residual 연산을 수행한다. `downsample=True` 이면 `downsize` 로 x의 크기를 F(x)와 맞춘 뒤 더하고, `downsample=False` 이면 채널 수가 다를 경우에만 `make_equal_channel` 을 적용한다.

Question 2: `class ResNet50_layer4(nn.Module)`

ResNet50_layer4는 아래 구조의 4계층 ResNet-50을 정의한다.

Layer	구조	출력 크기
Layer 1	7×7 Conv(64, stride=2) → BN → ReLU → MaxPool(stride=2)	8×8×64
Layer 2	ResidualBlock ×2 (False) + ×1 (True)	4×4×256
Layer 3	ResidualBlock ×3 (False) + ×1 (True)	2×2×512

Layer	구조	출력 크기
Layer 4	ResidualBlock ×6 (False)	2×2×1024
AvgPool	2×2	1×1×1024
FC	1024 → 10	10

`__init__(self, num_classes=10)`

CIFAR-10의 클래스 수가 10개이므로 `num_classes=10` 으로 설정한다.

Layer 1:

```
self.layer1 = nn.Sequential(
    nn.Conv2d(3, 64, 7, 2, 3),
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(3, 2, 1)
)
```

CIFAR-10 입력(32×32×3)이 stride=2 conv를 통과해 16×16×64가 되고, MaxPool(stride=2) 후 8×8×64가 된다.

Layer 2:

```
self.layer2 = nn.Sequential(
    ResidualBlock(64, 64, 256, False),
    ResidualBlock(256, 64, 256, False),
    ResidualBlock(256, 64, 256, True)
)
```

처음 두 블록은 downsample 없이 8×8×256을 유지하고, 마지막 블록에서 stride=2 다운샘플링으로 4×4×256을 출력한다.

Layer 3:

```
self.layer3 = nn.Sequential(
    ResidualBlock(256, 128, 512, False),
    ResidualBlock(512, 128, 512, False),
    ResidualBlock(512, 128, 512, False),
    ResidualBlock(512, 128, 512, True)
)
```

세 블록은 downsample 없이 4×4×512를 유지하고, 마지막 블록에서 다운샘플링으로 2×2×512를 출력한다.

Layer 4:

```
self.layer4 = nn.Sequential(
    ResidualBlock(512, 256, 1024, False),
    ResidualBlock(1024, 256, 1024, False),
    ResidualBlock(1024, 256, 1024, False),
    ResidualBlock(1024, 256, 1024, False),
    ResidualBlock(1024, 256, 1024, False),
)
```

```
ResidualBlock(1024, 256, 1024, False)
)
```

6개 블록 모두 downsample 없이 $2 \times 2 \times 1024$ 를 유지한다.

FC & AvgPool:

```
self.fc = nn.Linear(1024, 10)
self.avgpool = nn.AvgPool2d(2, 2)
```

AvgPool로 $2 \times 2 \times 1024 \rightarrow 1 \times 1 \times 1024$ 로 줄인 뒤, FC layer에서 10개 클래스 점수를 출력한다.

forward(self, x)

```
out = self.layer1(x)
out = self.layer2(out)
out = self.layer3(out)
out = self.layer4(out)
out = self.avgpool(out)
out = out.view(out.size()[0], -1)
out = self.fc(out)
return out
```

Layer1 → Layer2 → Layer3 → Layer4 → AvgPool → flatten → FC 순으로 순전파를 수행한다.

main.py

CIFAR-10 데이터셋을 불러오고 VGG-16 또는 ResNet-50 모델을 학습한 뒤 테스트 정확도를 평가한다.

1. 전처리

```
transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.201
0))),
])
transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.201
0))),
])
```

학습 데이터에는 RandomCrop과 RandomHorizontalFlip으로 data augmentation을 적용하고, 테스트 데이터에는 정규화만 적용한다. 이후 DataLoader로 batch_size=100의 미니배치 단위로 로드한다.

2. 학습 및 평가

하이퍼파라미터는 `num_epochs=1`, `learning_rate=0.001`로 설정하고 손실 함수는 `CrossEntropyLoss`, optimizer는 `Adam`을 사용한다.

각 배치마다 순전파 → 손실 계산 → `optimizer.zero_grad()` → 역전파(`loss.backward()`) → 가중치 갱신(`optimizer.step()`) 순으로 진행하며, 100 step마다 평균 손실을 출력한다.

학습 완료 후 `model.eval()` 과 `torch.no_grad()` 를 사용해 평가 모드로 전환하고 테스트 정확도를 계산한다.

3. 실험 결과

모든 실험은 GPU(CUDA) 환경에서 CIFAR-10 데이터셋으로 학습 및 테스트하며, `CrossEntropyLoss`와 `Adam` optimizer(lr=0.001)를 사용해 1 epoch 학습한다.

3.1 VGG-16

```
... VGG-16 체크포인트 로드 완료
Epoch [1/1], Step [100/500] Loss: 0.1953
Epoch [1/1], Step [200/500] Loss: 0.1777
Epoch [1/1], Step [300/500] Loss: 0.1781
Epoch [1/1], Step [400/500] Loss: 0.1832
Epoch [1/1], Step [500/500] Loss: 0.1861

VGG-16 Test Accuracy: 86.70 %
```

실행 결과 테스트 정확도가 **86.70%**로 85% 이상을 만족하는 것을 확인하였다. Loss가 0.19 수준에서 안정적으로 수렴하는 것을 볼 수 있으며, 이는 250 epoch 사전 학습된 체크포인트를 기반으로 추가 학습한 결과이다.

3.2 ResNet-50

```
사용 디바이스: cuda:0
ResNet-50 체크포인트 로드 완료
Epoch [1/1], Step [100/500] Loss: 1.5427
Epoch [1/1], Step [200/500] Loss: 1.0692
Epoch [1/1], Step [300/500] Loss: 0.8825
Epoch [1/1], Step [400/500] Loss: 0.7848
Epoch [1/1], Step [500/500] Loss: 0.7273

ResNet-50 Test Accuracy: 81.37 %
```

실행 결과 테스트 정확도가 **81.37%**로 80% 이상을 만족하는 것을 확인하였다. VGG-16에 비해 Loss 초기값이 높고 감소 폭이 크게 나타나는데, 이는 285 epoch 체크포인트임에도 불구하고 bottleneck 구조 특성상 학습 초기 수렴 속도가 상대적으로 느리기 때문으로 분석된다.

4. 소감

이번 과제를 통해 VGG-16과 ResNet-50의 구조를 직접 코드로 구현하며 두 모델의 설계 철학 차이를 체감할 수 있었다. VGG-16은 단순히 3×3 conv를 깊게 쌓는 구조인 반면, ResNet-50은 skip connection을 통해 깊은 네트워크에서도 gradient가 잘 흐를 수 있도록 설계되었다는 점이 인상적이었다. 특히 다운샘플링 여부에 따라 skip connection 처리를 달리해야 하는 부분에서 residual block의 구조를 더 깊이 이해할 수 있었다. 또한 실험 결과에서 VGG-16과 ResNet-50의 Loss 감소 패턴이 다르게 나타나는 것을 확인하며, 모델 구조가 학습 양상에 미치는 영향을 직접 확인할 수 있었다.